D02｜每次再试，都要花同一份预算

本章只读取容量，不创建或占用名额。**值得重试、能够安全重复、时间与次数仍够**，三者同时成立才进入下一次；一项成立不能替另两项作保证。

一次等待怎样花到总预算里？

当前问题	关系或短用法	记到什么程度
怎么算这次最多还能等多久？	`deadline = started + total`；`remaining = deadline - now`；`allowed = min(per_attempt, remaining)`。每次调用前重新计算余额。	能跟着经过时间算余额；用单调时钟计时，日期、时区和校时不应改变预算。
backoff 是否另算？	第一次超时 1 秒，暂停 0.2 秒，第二次成功 0.1 秒，共花 1.3 秒。暂停也从同一份总预算支付。	**重试之间的等待不免费。** 暂停后还须给下一次留下时间。
还有 0.3 秒为什么停止？	2.5 秒总预算，两次均耗时 1 秒，中间等 0.2 秒；时间尚余，最多 2 次已用完。	“最多 2 次”包含首次；时间和次数是两道停止条件。
允许重试，为什么没有第二次？	总预算 1.1 秒，首次超时后只剩 0.1 秒，付不起 0.2 秒 backoff；停止且不消费后面的成功剧本。	能从余额解释零新增调用，不先睡完再检查。
发起时没超期，返回时仍可拒绝吗？	应答可能到达 deadline 之后，程序再检查并归为迟到；“收到合法结果”与“仍在接纳期限内”分开。	认清前后两次检查；拒绝迟到结果不代表取消远端执行。

本例总预算 1.25 秒时，第一次超时与 backoff 后第二次约剩 0.05 秒；fake 故意让 success 花 0.1 秒，触发 `late_response`。数值帮助看关系，不是通用配置建议。

回看：「把总共允许等待的时间先写出来」「用一个可推进的时钟看见控制路线」。

该不该再试，由谁决定?

当前问题	关系或短用法	记到什么程度
500 一律重试、401 原样再试，有依据吗?	本章容量合同仅允许 timeout 再试；401、500 和坏内容立即停。D05 的创建合同另明确允许一组 5xx，不能反套到这里。	**重试许可属于当前操作与 provider 合同。** 不由“服务器错误”几个字自动决定。
GET 就一定能放心重复吗?	本章实际只读容量，才符合安全读取；把创建预约错误地写在 GET 中，不会被方法名变安全。	操作效果与声明语义都要看；有副作用的创建转 D03。
`httpx.Timeout(1.0)` 是整项业务一秒硬停止吗?	它分别约束 connect/read/write/pool 等阶段等待；应用 deadline 控制继续调用与接纳结果，不能凭空中断尚未返回的同步调用。	**阶段等待与端到端取消分开。** 真实硬取消需运行环境机制及外部效果证据。
上下两层都最多尝试两次，会调用几次?	外层两轮、底层每轮两次，可能累计四次。本例 sender 一次只发送一次，策略层独自控制 retry。	明确次数由一处负责，日志 attempt 要对应实际发送。
fake clock、jitter 各解决什么?	fake clock 推进数字，验证预算和未消费剧本；jitter 随机错开大量同时重试，但也花总预算。	工具实现查用；fake 计时不是真实等待测量，抖动不增加承诺时间。

容量字段要排除布尔值：Python 的 `True` 也满足 `isinstance(True, int)`，但不能冒充一个预约名额。完整 sender 与时钟程序查正文；日志认熟 attempt、耗时、类别即可，不记录凭据或整个回复。

回看：「哪些失败值得花第二次机会?」「一个 timeout 数字为什么不是整项工作的硬秒表?」。